

C-Blondel: An Efficient Louvain-Based Dynamic Community Detection Algorithm

Mahsa Seifikar^{ID}, Saeed Farzi^{ID}, and Masoud Barati

Abstract—One of the most interesting topics in the scope of social network analysis is dynamic community detection, keeping track of communities' evolutions in a dynamic network. This article introduces a new Louvain-based dynamic community detection algorithm relied on the derived knowledge of the previous steps of the network evolution. The algorithm builds a compressed graph, where its supernodes represent the detected communities of the previous step and its superedges show the edges among the supernodes. The algorithm not only constructs the compressed graph with low computational complexity but also detects the communities through the integration of the Louvain algorithm into the graph. The efficiency of the proposed algorithms is widely investigated in this article. By doing so, several evaluations have been performed over three standard real-world data sets, namely Enron Email, Cit-HepTh, and Facebook data sets. The obtained results indicate the superiority of the proposed algorithm with respect to the execution time as an efficiency metric. Likewise, the results show the modularity of the proposed algorithm as another effectiveness metric compared with the other well-known related algorithms.

Index Terms—Community detection, dynamic community detection, large-scale network analysis, Louvain algorithm.

NOMENCLATURE

Symbol	Definition
T	Number of snapshots.
G_t	Graph in snapshot t .
V_t	Set of nodes in snapshot t .
E_t	Set of edges in snapshot t .
d_u	Degree of node u .
C_u	Community of node u .
N_u	Node u 's neighborhood.
$\bar{d}_c$	Average degree of nodes within community c .
cc	Clustering coefficient of a graph.
A_{uv}	Number of edges between node u and v .
n	Total number of graph nodes.
m	Total number of graph edges.
m_c	Number of edges within community c .

I. INTRODUCTION

THE exponential growth of the social network users leads to a large number of activities performed by the users as

well as a massive amount of information exchanging within the network [1]. Given the analysis of such networks, hidden information can be extracted. Moreover, the social patterns related to users' interactions, which can be employed to solve a wide range of real-world problems, e.g., political, economic, social issues, and so on, can be achieved [2].

According to the network structure, a node denotes a user, and an edge depicts an interaction between two users. A community expresses the mutual interest among its members, e.g., users with common political interests in twitter form a community [3]–[5]. A social community is one of the fundamental concepts in social network studies defined as a group of people who have dense connections among each other and sparse relationships with others people outside the community [6]–[9]. Dynamic community detection algorithms have paid a considerable amount of attention in the last decade [1], [10]–[14]. Besides, the Blondel method [15] has attracted many researchers because of enhanced performance and its logarithmic computational complexity.

There is a considerable evolution in the real-world social networks over a long period of time [14]. During the evolution of the network (e.g., Instagram, Facebook, and Twitter), users and its connections may appear or disappear. The time-dependent nature of social networks makes researchers focus on dynamic community detection algorithms, generally categorized into two groups as follows.

1) The first group runs a specific community detection algorithm for all snapshots and then maps the detected communities of two consecutive snapshots. These algorithms usually achieve high modularity in a very time-consuming manner. Indeed, dynamic community detection algorithms are faced with a massive number of nodes and edges as well as a large number of snapshots. Therefore, low execution time is as important as achieving high modularity [16]–[21].

2) The second group employs previous snapshots communities in order to detect the current snapshot communities. In other words, they utilize historical information [1], [11]–[14] to reduce execution time [1], [11]–[14], [22]–[25].

This article introduces a new dynamic community detection algorithm using historical information of networks. The proposed algorithm in this article, hereafter called C-Blondel, is considered for the improvement of the run time complexity. C-Blondel applies the Louvain algorithm over a compressed graph derived from the previous snapshot. C-Blondel is faster than the Louvain algorithm because of the smaller size of the compressed graph, constructed from particular nodes called supernodes and superedges. A supernode represents a previous

Manuscript received June 12, 2019; revised October 17, 2019 and December 14, 2019; accepted December 21, 2019. Date of publication February 4, 2020; date of current version April 3, 2020. (Corresponding author: Saeed Farzi.)

Mahsa Seifikar and Saeed Farzi are with the Faculty of Computer Engineering, K. N. Toosi University of Technology, Tehran 16317-14191, Iran (e-mail: m.seifikar@email.kntu.ac.ir; saeedfarzi@kntu.ac.ir).

Masoud Barati is with the School of Computer Science and Informatics, Cardiff University, Cardiff CF10 3AT, U.K. (e-mail: baratim@cardiff.ac.uk). Digital Object Identifier 10.1109/TCSS.2020.2964197

snapshot detected community and there is no destructive node inside of it. The destructive node is defined as a highly influential node in a community which the lack of it makes blow up the corresponding community. In order to recognize destructive nodes, a simple and effective heuristic approach based on a degree centrality is introduced. A node is called a destructive node if the degree centrality of the node is higher than the average degree of the corresponding community nodes. In C-Blondel, all network changes, including appearing a node, appearing an edge, disappearing a node, and disappearing an edge, are considered. However, all changes are handled by the action of disappearing a node. During the action, when a destructive node is eliminated, its community is split into subcommunities. After considering all network changes, the rest communities and subcommunities are converted into supernodes, and edges between them refer to the superedges. Finally, the compressed graph is generated.

In experimental results, two main scenarios have been considered: first one with the purpose of investigating the impact of C-Blondel parameter values on community quality and speed up and the second one with the aim of comparing C-Blondel with other well-known algorithms, which is S-Blondel [16] and D-Blondel [1] over three real-world standard data sets, i.e., Enron Email data set [26], Cit-HepTh data set [27], and Facebook data set [28]. Enron Email data set is built from the e-mail exchanges among users of Enron enterprise [26]. Cit-HepTh is a collaboration network extracted from e-print arXiv and covers all citations of arXiv article repository [27]. Facebook contains the relationship information among the users [28].

The results indicate that C-Blondel outperforms S-Blondel and D-Blondel with regard to the execution time complexity, whereas the community quality of C-Blondel, which is measured by modularity metric, is as good as other aforementioned algorithms.

The rest of this article is organized as follows. In Section II, the related works are reviewed, and in Section III, the detail of C-Blondel is presented. In Section IV, the data sets and experiments are discussed. Finally, some conclusions are drawn in Section V.

II. RELATED WORK

Given various kinds of information used by community detection algorithms, they are classified into global and local algorithms [29]. The former utilizes the entire network structure information, whereas the latter exploits only partial information of the network structure. Although the global algorithms produce accurate outputs through nonscalable manners, the local ones are less accurate. Moreover, they are tractable for large-scale networks [29].

Indeed, maximizing modularity as a quality metric of community detection algorithms needs information of all vertices of the given network. Therefore, the whole network topology information must be taken into consideration. Some modularity-based algorithms, such as the Girvan–Newman algorithm [3] or a centrality-based algorithm [30], are

classified into global algorithms. However, these methods are not scalable and cannot be successfully executed on large networks.

In contrast, the local algorithms leverage the information of the nodes' neighbors to find communities. Generally, these methods maximize modularity thereby greedy strategies and they may get stuck in local optima, and subsequently cannot find good community structure. The Blondel algorithm [15] is a well-known modularity-based algorithm that is grouped in the local algorithms.

Since dynamic community detection algorithms deal with large nodes, edge counts, and a large number of snapshots, its computational time becomes a main significant factor. Hence, some dynamic community detection algorithms exploited local algorithms, such as Louvain supplemented by heuristic approaches to escape from local optima.

We review both global and local algorithms as follows.

A. Global Algorithms

Yang *et al.* [31] introduced a dynamic community detection algorithm relied on matrix factorization. First, for each snapshot, a weighted network was constructed through computing the embeddedness and dispersion among every pair of nodes to calculate the strength of the relations. Then, a node strength matrix representing the connection strength of a node pair was built. Finally, communities' structures were discovered by combining the topological information at the previous snapshot and the nonnegative matrix factorization. The accuracy of their proposed algorithm was reported for two synthetic networks. One of the main disadvantages of their approach was nonscalability. In fact, the proposed algorithm was very time consuming.

Liu *et al.* [32] proposed a global spectral community detection algorithm based on the eigenvector smoothing algorithm, merging knowledge across a collection of networks. The degree of correction and evolutionary spectral clustering methods were two main components of their proposed algorithm. The algorithm was evaluated on a novel data set in which gene expression has been computed during a wide range of developmental periods in rhesus monkey brains.

In [33], a game theory-based community detection algorithm was presented to detect communities in dynamic networks. Every node in the network was considered as a selfish agent who could choose a set of actions, i.e., join, leave, or community switch in order to optimize a utility function.

B. Local Algorithms

Zhou *et al.* [34] presented a dynamic community detection algorithm based on multiobjective discrete bat algorithm aiming at overcoming the parameter selection limitation and enhancing the quality of discovering communities. In order to preserve the diversity of the population, two novel turbulence and mutation operators were introduced.

Zhao *et al.* [35] presented an incremental dynamic community detection algorithm by handling subgraphs. First, they identified four types of essential incremental elements and

then introduced different updating strategies for the proposed algorithm.

Wu *et al.* [36] introduced an evolutionary matrix-based community detection algorithm for dynamic networks. The matrix tracked the network evolutions via combining the community structure of the previous snapshot with the network structure of the current snapshot. First, a weighted network was generated, and then, an edge density-based overlap community detection algorithm was undertaken on it. Finally, the edge communities were reformed to the node communities.

To overcome the main challenges that the most of the evolutionary-based dynamic community detection algorithms encountered with, Messaoudi and Kamel [37] proposed a bat algorithm for discovering communities in dynamic networks optimizing modularity and normalized mutual information (NMI) as objective functions.

Greene *et al.* [16] proposed the first general model for dynamic community detection adopted with most community detection algorithms. A community detection algorithm was applied to the individual network snapshot, and then, the Jaccard similarity. (Jaccard, 1912) was used to map the communities of two consecutive snapshots in a time-consuming manner.

Nguyen *et al.* [14] proposed an incremental Louvain-based algorithm in order to trace communities along time. They partitioned network changes into four groups of appearing a node or an edge and disappearing a node or an edge. First, they used the Louvain algorithm [15]) to detect initial community structure. Then, they handled network changes by some modularity-based rules in next snapshots. Similar to [14], Cordeiro *et al.* [13] considered all four categories of network changes, but they applied a modified version of the Louvain algorithm and different modularity-based rules.

Xin *et al.* [24], [25] introduced two novel algorithms based on random walk algorithm, called ARWS and ANRW, able to take overlapping communities into account. Similar to [13] and [14], all network changes were considered separately. However, the main drawback was over parametrization.

Niu *et al.* [12] optimized an objective function, including community quality criteria and computational cost using a genetic-based algorithm, called L-DMGA. The label propagation algorithm [20] was used to detect community in the first snapshot. In the next snapshots, an individual node was considered as a gene, and then, the algorithm generated an initial population that represents the communities of a network. Through a genetic algorithm process, an optimal partitioning of nodes was achieved. The modularity and NMI¹ criteria were used to compute the quality of communities and temporal costs, respectively. The main advantage was linear time complexity, whereas there were some drawbacks, such as inefficiencies in large networks and finding appropriate parameter values.

He *et al.* [1] proposed a two-step division and agglomeration clustering algorithm based on the Louvain algorithm, hereafter called D-Blondel. First, in the division step, the tendency of each node toward its neighbors has computed; then, nodes

from the different communities were excluded from the corresponding community based on their tendency. If a node is extracted from its community, the relevant community is broken into subcommunities and a subcommunity, considered as a supernode in the main graph. Second, in the agglomeration step, the Louvain algorithm was applied to the graph, obtained in the division step. In order to keep track of changes among different consecutive snapshots, the intersection of the constituent module of the supernodes was used. Since the number of modules is dramatically less than the number of nodes, the speed of tracking was a lot more by a meaningful difference. Agarwal *et al.* [11] reported the first dynamic community detection algorithm, called DyPerm, optimizing a vertex-centric community metric, called permanence. They used NMI to measure how the communities are accurate.

As the change of all network nodes is a time-consuming task, C-Blondel reformulated (appearing|disappearing) (node|edge) to disappearing node action from the network. On the other hand, similar to [1], C-Blondel contains the division and agglomerate steps, but instead of computing the tendency of all nodes, C-Blondel just computed tendency of the destructive nodes. In the division step, the graph was partitioned based on the tendency of the destructive node. In agglomerative steps, C-Blondel applied the Louvain algorithm to detect communities.

III. METHOD

In order to explain the proposed algorithm, first, formal definition and then objective function are described in more detail.

A. Formal Definition

All notations and abbreviations used in this article are listed under the Nomenclature section.

1) *Definition Dynamic Social Network:* It is defined as a set of graphs $G = \{G_1(V_1, E_1), G_2(V_2, E_2), \dots, G_t(V_t, E_t), \dots, G_T(V_T, E_T)\}$ at successive intervals where $G_t(V_t, E_t)$ is an undirected weighted graph that shows a snapshot of the network [10].

2) *Definition Community Detection:* It is a division $C(G_t) = \{c_{t1}, c_{t2}, \dots, c_{tk}\}$ of G_t to k subgraph such that $V = \bigcup_{i=1}^k C_i$ [6].

3) *Definition Dynamic Community Detection:* It is defined as finding sequences of similar communities in different snapshots in order to track the evolution of the network communities [10].

4) *Definition Compressed Graph:* A compressed graph, $G^H(V^H, E^H)$, which is constructed from graph $G_t(V_t, E_t)$, is undirected and weighted graph, where $V^H = \{V_1^H, V_2^H, \dots, V_n^H\}$ contains supernodes and $E^H = \{(V_i^H \in V_j^H) | V_i^H \in V^H\}$ contains superedges. A supernode V_i^H is a community or a subcommunity of the graph $G_{t-1}(V_{t-1}, E_{t-1})$ and a superedge is an edge between two supernodes.

5) *Definition Destructive Node:* It is defined as a highly influential node of the community. Its elimination makes blow up the corresponding community.

¹NMI

B. Objective Function

In order to evaluate the quality of communities, the well-known metric called modularity is used [38] as follows:

$$Q = \frac{1}{2m} \sum_{vw} \left[A_{vw} - \frac{k_v k_w}{2m} \right] \delta(c_v, c_w) \quad (1)$$

where k_v and k_w represent the degree of the nodes v and w , respectively. $\delta(\cdot)$ function is the Kronecker delta function, which is “1” if c_v is equal to c_w and otherwise equals “0.” Higher value for modularity means better community structure [13].

C. Proposed Method

In order to detect communities of a graph $G_t(V_t, E_t)$ in a snapshot t , there are two distinct ways. The simplest and time-consuming way integrates the Louvain algorithm into the graph $G_t(V_t, E_t)$. The efficient way is to consider the detected communities of the previous snapshot, as well as network changes occurred during snapshot $t - 1$ and t , in order to detect communities of $G_t(V_t, E_t)$ in snapshot t .

To encapsulate and summarize historical information of a dynamic social network in the snapshot t , a compressed graph $G_t^H(V_t^H, E_t^H)$ is constructed from the graph $G_t(V_t, E_t)$ and the previous snapshot communities. The compressed graph $G^H(V^H, E^H)$ is more compact and contains more information about the current snapshot communities than the original graph.

The size of $G_t^H(V_t^H, E_t^H)$ is smaller than the original graph $G_t(V_t, E_t)$ because of $|V_t| \gg |V_t^H|$ and $|E_t| \gg |E_t^H|$; Therefore, the run time complexity of the Louvain algorithm over $G_t^H(V_t^H, E_t^H)$ is lower than running on $G_t(V_t, E_t)$. Algorithm 1 illustrates a pseudocode of C-Blondel.

Algorithm 1 C-Blondel Algorithm

```

1: Input:
2:  $G_{t-1} V_{t-1} E_{t-1}$  : a graph of snapshot  $t - 1$ 
3:  $G_t V_t E_t$  : a graph of snapshot  $t$ 
4:  $C(G_{t-1})$  : a set of detected communities of  $G_{t-1}$ 
5: Output:
6:  $C(G_t)$  : A set of detected communities of  $G_t$ .
7: function C-Blondel( $G_{t-1} G_t, C(G_{t-1})$ )
8: begin
9:    $G_t^H V_t^H E_t^H \leftarrow$ 
     ConstructCompressedGraph( $G_{t-1} G_t, C(G_{t-1})$ )
10:   $C(G_t) \leftarrow LA(G_t^H V_t^H E_t^H)$ 
11: end

```

where $LA(\cdot)$ function runs the Louvain algorithm and *ConstructCompressedGraph*($\cdot$) function creates a compressed graph for snapshot t via considering $C(G_{t-1})$ and network changes during snapshot $t - 1$ and t .

1) *Compressed Graph*: Network changes are classified into four groups, comprising of (appearing|disappearing) (nodes|edges). The main idea is based on a fact that including and excluding nodes and edges from the graph have a high impact on their closed neighbors. Therefore, in order to construct a compressed graph, all affected nodes are removed

from their communities and considered as a supernode. Thus, any change in the network has been observed as disappearing a node from its community, which possibly blows up a community. Here, the most important question that must be answered is what sort of nodes destroys their communities. In order to answer the question, Lemma 1 is considered.

Lemma 1: Node u is a destructive node if $d_u \geq p$.

Proof: Assume that community C can be divided into $X_1, X_2, \dots, X_k$ disjoint communities.

Recall that the modularity is

$$Q = \frac{1}{2m} \sum_{vw} \left[A_{vw} - \frac{k_v k_w}{2m} \right] \delta(c_v, c_w). \quad (2)$$

If δ function is removed, therefore, the modularity is concluded as follows [14]:

$$Q = \sum_{c \in C} \left[\frac{m_c}{m} - \frac{d_c^2}{4m^2} \right]. \quad (3)$$

For individual community, we have

$$Q_c = \frac{m_c}{m} - \frac{d_c^2}{4m^2}, \quad Q_{X_i} = \frac{m_i}{m} - \frac{d_i^2}{4m^2} \quad (4)$$

and

$$m_c = \sum_{i=1}^k m_i + \sum_{i < j} e_{ij}. \quad (5)$$

If removing node, u , with degree p from X_1 led to community C blow up, then we have

$$Q_c < \sum_{i=1}^k Q_{X_i} \quad (6)$$

$$\Leftrightarrow \frac{\sum_{i=1}^k m_i + \sum_{i < j} e_{ij} - p}{m - p} - \frac{\left(\sum_{i=1}^k d_i - p \right)^2}{4(m-p)^2} < \frac{m_1 - p}{m - p} - \frac{(d_1 - p)^2}{4(m-p)^2} + \sum_{i=2}^k \left(\frac{m_i}{m - p} - \frac{d_i^2}{4(m-p)^2} \right) \quad (7)$$

$$\Leftrightarrow \frac{\sum_{i=1}^k m_i + \sum_{i < j} e_{ij} - p}{m - p} - \frac{\left(\sum_{i=1}^k d_i - p \right)^2}{4(m-p)^2} < \frac{\sum_{i=1}^k m_i - p}{m - p} - \frac{(d_1 - p)^2}{4(m-p)^2} - \sum_{i=2}^k \frac{d_i^2}{4(m-p)^2} \quad (8)$$

$$m_c = \sum_{i=1}^k m_i + \sum_{i < j} e_{ij} \quad (9)$$

$$\Leftrightarrow \sum_{i < j} e_{ij} < \frac{\sum_{i < j} d_i d_j + p \left(d_1 - \sum_{i=1}^k d_i \right)}{2(m - p)} \quad (10)$$

$$\Leftrightarrow \frac{2M \sum_{i < j} e_{ij} - \sum_{i < j} d_i d_j}{2 \sum_{i < j} e_{ij} + d_1 - \sum_{i=1}^k d_i} < p. \quad (11)$$

Given Lemma 1, it is observed that if the node degree is greater than a threshold, the community associated with the node collapses. Because a precise calculation of this threshold

Algorithm 2 Construct Compressed Graph Function

```

1: Input:
2:  $G_{t-1}V_{t-1}E_{t-1}$  : a graph of snapshot  $t-1$ 
3:  $G_tV_tE_t$  : a graph of snapshot  $t$ 
4:  $C(G_{t-1})$  : a set of detected communities of  $G_{t-1}$ 
5: Output:
6:  $G_t^H V_t^H E_t^H$  : a compressed graph of snapshot  $t$ 
7: function ConstructCompressedGraph( $G_{t-1}G_t, C(G_{t-1})$ )
8:   begin
9:      $\Delta G \leftarrow \text{PullOutChanges}(G_{t-1}G_t)$ 
10:    for each  $ch \in \Delta G$ 
11:      if  $ch.type \text{ equal } \text{"Removing Node"}$  then
12:         $u \leftarrow ch.node$ 
13:         $\langle V_t^H E_t^H \rangle \leftarrow \text{RemoveNode}(u, C_u)$ 
14:      if  $ch.type \text{ equal } \text{"Removing Edge"}$  then
15:         $\langle uv \rangle \leftarrow ch.edge$ 
16:        if  $C_u \text{ equal to } C_v$  then
17:           $\langle V_t^H E_t^H \rangle \leftarrow \text{RemoveNode}(u, C_u)$ 
18:           $\langle V_t^H E_t^H \rangle \leftarrow \text{RemoveNode}(v, C_v)$ 
19:        if  $ch.type \text{ equal } \text{"Adding Node"}$  then
20:           $u \leftarrow ch.node$ 
21:           $V_t^H \leftarrow u$ 
22:          for each  $nb \in N_u$ 
23:            if  $C_{nb} \text{ not null}$  then
24:               $\langle V_t^H E_t^H \rangle \leftarrow \text{RemoveNode}(nb, C_{nb})$ 
25:          end for
26:        if  $ch.type \text{ equal } \text{"Adding Edge"}$  then
27:           $\langle u, v \rangle \leftarrow ch.edge$ 
28:           $E_t^H \leftarrow \langle uv \rangle$ 
29:          if  $C_u \text{ not equal to } C_v$  then
30:             $\langle V_t^H E_t^H \rangle \leftarrow \text{RemoveNode}(u, C_u)$ 
31:             $\langle V_t^H E_t^H \rangle \leftarrow \text{RemoveNode}(v, C_v)$ 
32:          end if
33:    end for

```

is time consuming, a simple and effective heuristic solution is used based on the degree centrality. As much as the degree of a node u is higher than the average degree centrality of nodes belonging to its community, the node is known as highly destructive. In fact, the node u is destructive if $d_u \geq \alpha \bar{d}_{C_u}$, where $\bar{d}_{C_u}$ is the average degree of community C_u and α is called destruction parameter, being the only parameter of C-Blondel.

Algorithm 2 represents the pseudo of the construction of compressed graph. In line 9, *PullOutChanges*(-, -) function extracts all given network changes between snapshots $t-1$ and t . In line 13, *RemoveNode*(-, -) function removes a node from the community C , which its details are explained in Algorithm 3. In line 14, if particular edge $\langle u, v \rangle$ inside a community disappears, the *RemoveNode* function is applied to v and u . Otherwise, given [14], disappearing an edge between two different communities is caused by increasing the modularity of network. Also, in line 26, if particular edge $\langle u, v \rangle$ appears between two different communities, the *RemoveNode* function is applied to v and u . Otherwise, given [14], appearing an edge inside a community is caused

Algorithm 3 Remove a Node Method

```

1: Input:
2:  $u$ : a node of graph
3:  $C_u$ : community of node  $u$ 
4: Output:
5:  $\langle V_t^H E_t^H \rangle$ : A set of super nodes and super edges
6: function RemoveNode( $u, C_u$ )
7:   Begin
8:   if  $d_u \geq \alpha \bar{d}_{C_u}$  then
9:     for each  $sc \in LA(C_u)$ 
10:       $\langle V_t^H E_t^H \rangle \leftarrow$  convert  $sc$  to super nodes and
11:      super edges
12:     end for
13:   else then
14:      $\langle V_t^H E_t^H \rangle \leftarrow$  convert  $C_u$  to super nodes and
15:     super edges
16:   End

```

increasing in the modularity of the network. In line 19, if node u appears to the network in snapshot t , the *RemoveNode* function is applied to all its previous snapshot neighbors.

2) *Removing a Node*: Disappearing a node gives rise to the elimination of all its connections from a network. Also, this change may lead to blow up its community. According to Lemma 1, if a destructive node is disappeared, the corresponding community collapses.

Algorithm 3 shows the pseudocode of removing a node. In line 8, in order to detect destructive nodes, the average degree of community C_u is calculated. The distribution degree of nodes in large social networks is power law, so there are a large number of low degree nodes and few numbers of high degree nodes [39]. Therefore, the average degree of nodes is biased toward the nodes with fewer degrees. The average degree of nodes within each community is multiplied by the destruction parameter. Therefore, if the degree of node u is greater than $\alpha * \bar{d}_{C_u}$, then it is considered as a destructive node. The value of the destruction parameter depends on the social network degree distribution.

In line 9, *LA*(.) divides C_u into subcommunities. In line 11, individual subcommunity is considered as a set of supernodes and superedges. Moreover, if node u is not destructive, the corresponding community is considered as a supernode.

The execution time complexity of the C-Blondel algorithm is calculated as follows:

$$\begin{aligned}
O(\text{C-Blondel}) &= O(LA) + O(\text{CreateCompressedGraph}) \\
&= O(|V_t^H| \log |V_t^H|) + O(|\Delta G| * 2O(\text{RemoveNode})) \\
&= O(n_t^H \log n_t^H) + O(2 * |\Delta G| * n_t^c \log n_t^c). \quad (12)
\end{aligned}$$

IV. EXPERIMENTS

In order to evaluate C-Blondel, two scenarios with different objectives are followed. The aim of the first scenario is analyzing C-Blondel with respect to a destruction parameter. The purpose of the second one is comparing C-Blondel with

other well-known and popular algorithms in terms of execution time and community quality measured by modularity metric.

In the following, first, data sets and configuration have been described, and then, the comparison results have been analyzed.

The NMI and modularity are exploited to evaluate the performance of detecting communities in dynamic networks. NMI computes the similarity between community structure detected by the proposed algorithm and the ground-truth community structure [40]. In fact, it reveals the amount of detected community information in a corresponding network with known community structures. Also, modularity is, generally, defined by calculating the fraction of connections within communities minus the expected fraction of randomly distributed connections. In other words, modularity computes the strength of partitioning of a network into modules. High modularity implies that we have dense interconnections among nodes of modules but sparse intraconnections among nodes of different modules.

A. Data and Configuration

1) *Cit-HepTh Data Set*: Cit-HepTh data set is a citation network of articles extracted from arXiv² web portal. A node represents an article and an edge (i, j) depicts a citation from article i to article j . This data set has no information about the cited article outside the arXiv web portal [27].

2) *Enron Email Data Set*: Enron email data set contains roughly 0.5 million emails, gathered by the Federal Energy Regulatory Commission and preprocessed by scholars at MIT University. A node represents an e-mail address and an edge (i, j) depicts sending an e-mail from e-mail address i to e-mail address j [26].

3) *Facebook Data Set*: Facebook data set is a friendship network gathered from the Facebook New Orleans network. A node represents a user and an edge (i, j) , indicating user i , is a friend of user j [28].

B. Results

As mentioned earlier, in the first scenario, different values of the destruction parameter are evaluated for Cit-HepTh, Enron Email, and Facebook data sets. In the second scenario, modularity and the execution time of C-Blondel are compared with both D-Blondel [1] and S-Blondel [16]. In S-Blondel, the Louvain is performed on the original graph, while in the C-Blondel and D-Blondel algorithms, the Louvain algorithm is performed on the compressed graph of an individual snapshot.

Given the Moore visualization method [41], Cit-HepTh, Enron Email, and the Facebook, the network is partitioned into 31, 31, and 15 snapshots, respectively. Every snapshot aggregates data over a time interval [1]. All evaluation tasks have been run 30 times, and the average of modularity, execution time, and the communities' number have been reported.

1) *Cit-HepTh Data Set*: As it can be seen from Fig. 1, the distribution degree of the Cit-HepTh network is a power law. Given the use of the method proposed by

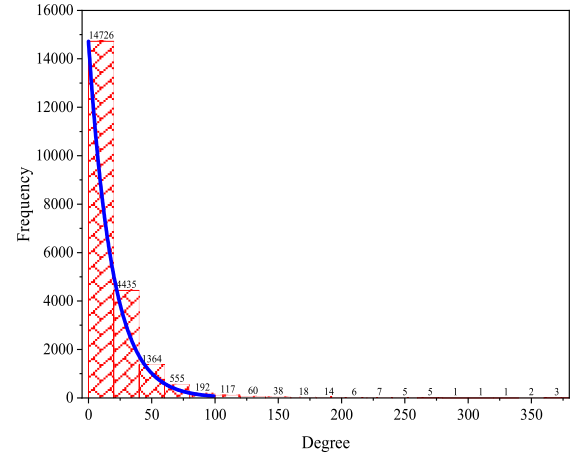


Fig. 1. Degree distribution of the Cit-HepTh data set.

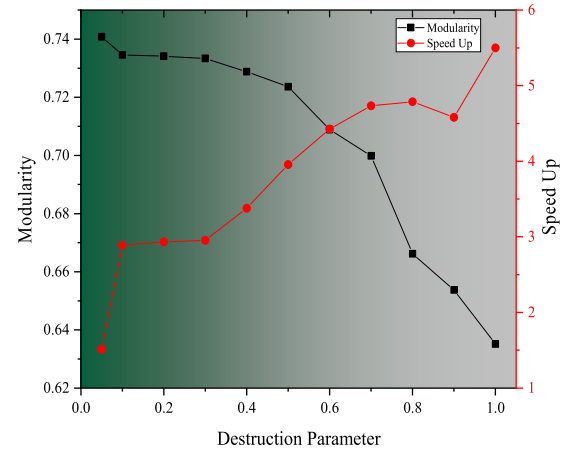


Fig. 2. Evaluating Destruction Parameter on the Cit-HepTh data set.

Clauset *et al.* [42], the coefficient α in this data set is estimated to be 4.17. Therefore, by increasing destruction parameter, the number of nodes inside supernodes is expected to increase. Consequently, the size of the compressed graph decreases sharply.

Fig. 2 shows that increasing destruction parameter reduces the modularity and enhances the execution time. If destruction parameter equals “0.6,” a balance between the community quality and the execution time is observed. Rising the community quality and falling the execution time are observed when the destruction parameter is less than “0.6” (right of Fig. 2). Falling the community quality and increasing the execution time are observed when the destruction parameter is more than “0.6” (left of Fig. 2). The best modularity is achieved at $\alpha = 0.1$, while the best execution time is achieved at $\alpha = 1$. Although the value of destruction parameter increases, the number of the destructive nodes decreases. Thus, a less number of the communities are blown up consequently. Also, the amount of the execution time increases. In other words, if the amount of destruction parameter decreases, the number of communities that are blown up increases. As a result, the algorithm runs over more nodes, and the quality is enhanced.

²<https://arxiv.org/>

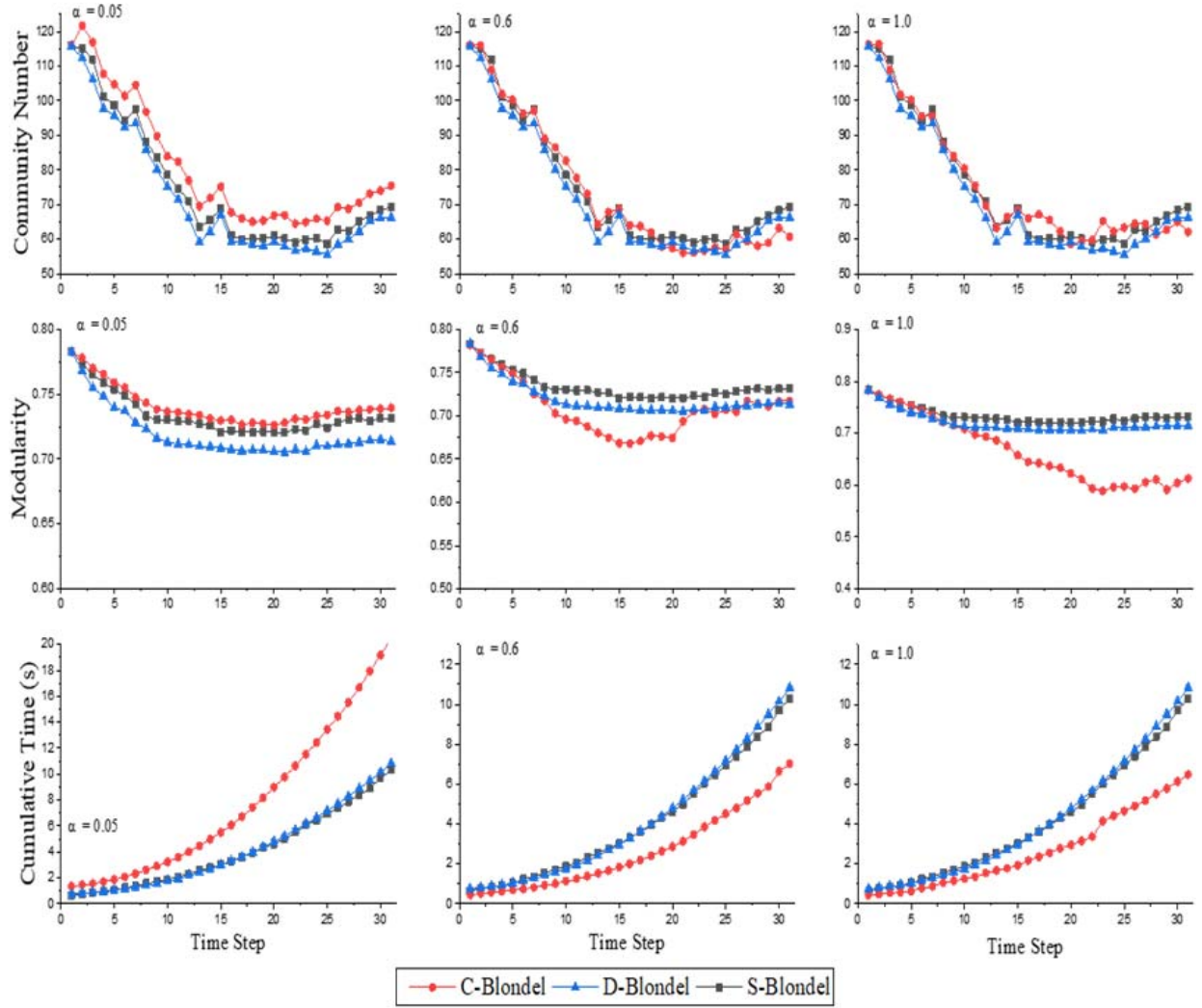


Fig. 3. Comparison of C-Blondel, D-Blondel, and S-Blondel in terms of execution time, modularity, and community number in the Cit-HepTh data set for the various destruction parameter.

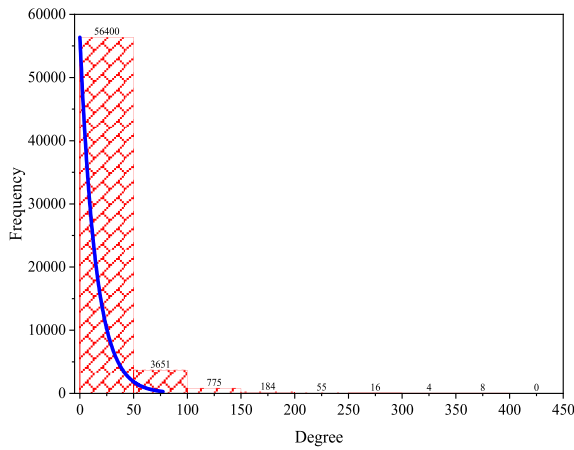


Fig. 4. Degree distribution of the Facebook data set.

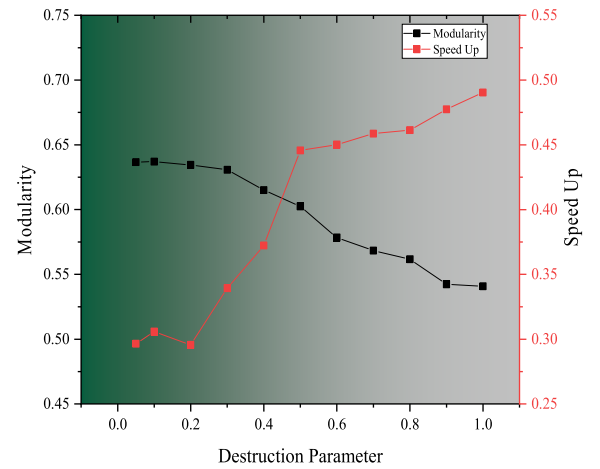


Fig. 5. Evaluating destruction parameter on the Facebook data set.

Fig. 3 shows the comparison among the results of S-Blondel, D-Blondel, and C-Blondel in terms of the number of communities, the modularity as community quality, and the execution time with three different destruction parameter values, i.e., $\alpha = 0.05$, $\alpha = 0.6$, and $\alpha = 1$. The number

of communities detected by C-Blondel is more than the number of communities detected by D-Blondel and S-Blondel at $\alpha = 0.05$. Also, C-Blondel keeps superiority than D-Blondel and S-Blondel in terms of modularity during all snapshots.

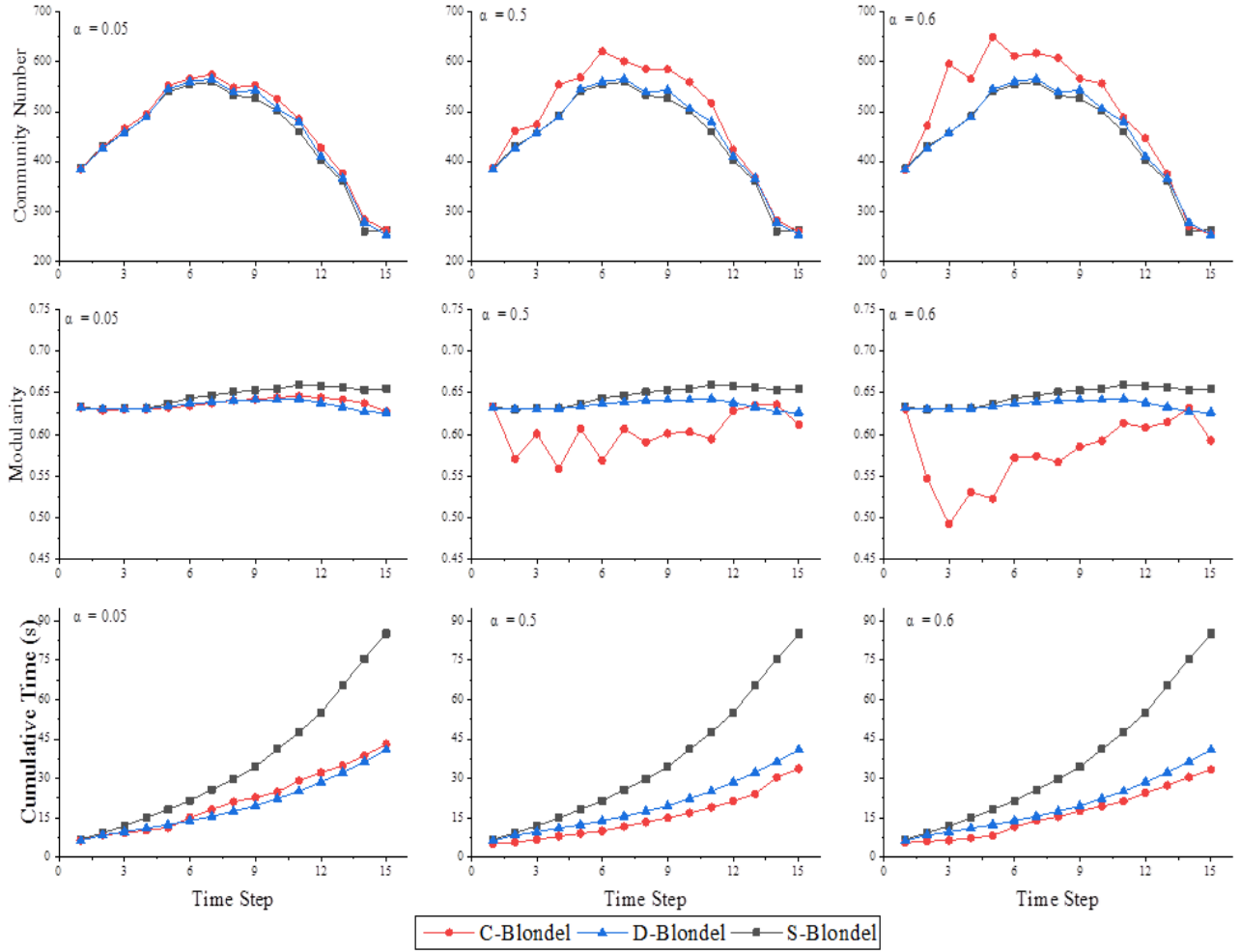


Fig. 6. Comparison of C-Blondel, D-Blondel, and S-Blondel in terms of execution time, modularity, and community number in the Facebook data set for the various destruction parameters.

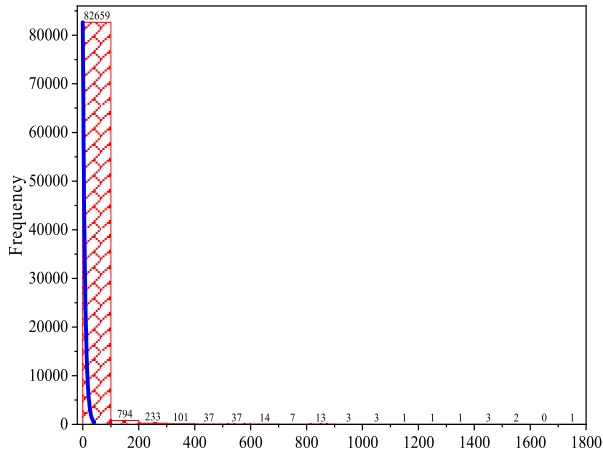


Fig. 7. Degree distribution of the Enron Email data set.

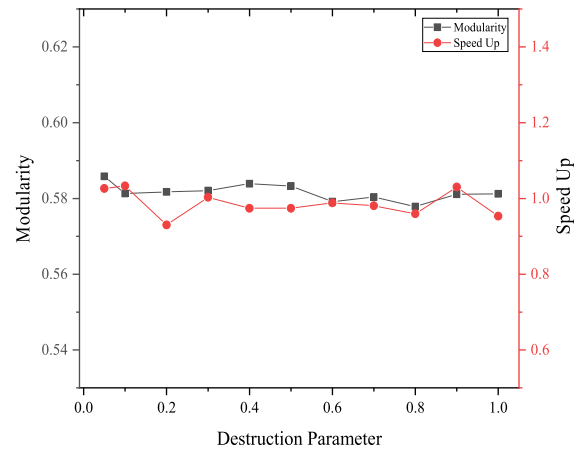


Fig. 8. Evaluating destruction parameter on the Enron Email data set.

On average, the C-Blondel achieves 0.007 and 0.02 points improvement than S-Blondel and D-Blondel in terms of modularity. However, the execution time of C-Blondel is slightly much more than S-Blondel and D-Blondel.

As it can be seen from Fig. 3, C-Blondel outperforms D-Blondel and S-Blondel at $\alpha = 0.6$ and $\alpha = 1.0$ according to

the execution time. Although C-Blondel achieved comparable modularity with two other methods at $\alpha = 0.6$, the modularity decreases sharply at $\alpha = 1.0$. C-Blondel achieves 0.106- and 0.124-s improvements compared with S-Blondel and D-Blondel with respect to the execution time at $\alpha = 0.6$ (in average). Furthermore, C-Blondel achieves 0.123- and

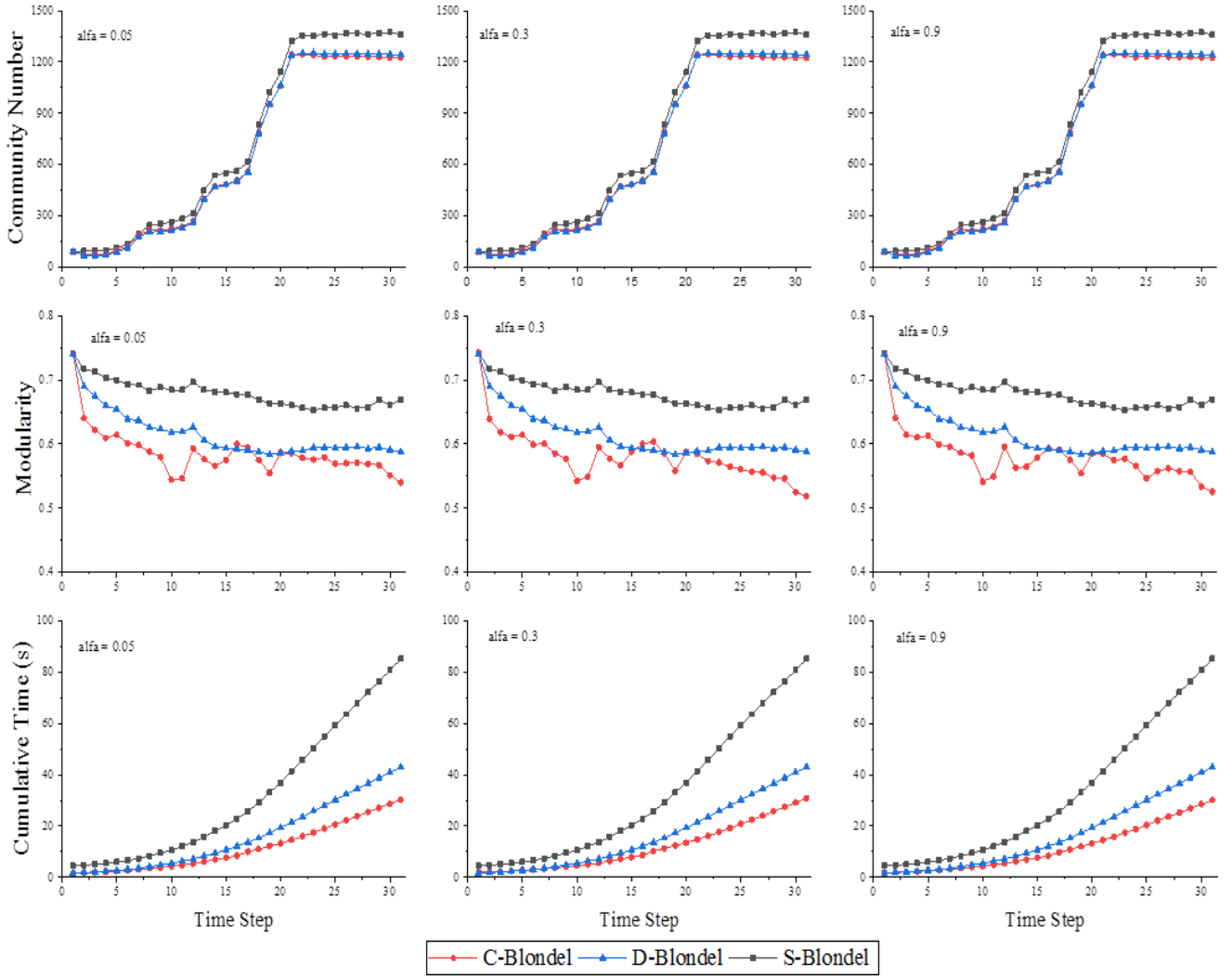


Fig. 9. Comparison of the C-Blondel, D-Blondel, and S-Blondel in terms of execution time, modularity, and community number in the Enron Email data set for the various destruction parameters.

0.14-s improvements compared with S-Blondel and D-Blondel with respect to the execution time at $\alpha = 1.0$ (in average).

2) *Facebook Data Set:* According to Fig. 4, the distribution degree of the Facebook network is power law. Given the use of the method proposed by Clauset *et al.* [42], the coefficient α in this data set is estimated to be 4.45. Therefore, by increasing the destruction parameter, the number of nodes inside the supernodes is expected to increase, and the size of the compressed graph decreases sharply.

As shown in Fig. 5, increasing the amount of destruction parameter reduces the modularity and increases the execution time. In $\alpha = 0.6$, it is observed a balance between the community quality and the execution time. Increasing the community quality and decreasing in the execution time are observed when the destruction parameter is less than 0.5 (right of Fig. 5). Decreasing in the community quality and increasing in the execution time are observed when the destruction parameter is more than 0.5 (left of Fig. 5). The best modularity is achieved at $\alpha = 0.1$, while the best execution time is achieved at $\alpha = 0.6$. If the amount of destruction parameter increases, the number of the destructive node decreases. Then,

the less number of the communities is blown up. Hence, the amount of the execution time increases. In other words, if the amount of destruction parameter decreases, the number of communities that are blown up increases. As a result, the algorithm runs over more nodes, and the community quality is enhanced.

Fig. 6 shows the comparison result of S-Blondel, D-Blondel, and C-Blondel in terms of the number of communities, the modularity (community quality), and the execution time with three different destruction parameter values, i.e., $\alpha = 0.05$, $\alpha = 0.5$, and $\alpha = 0.6$. The number of the communities detected by C-Blondel is more than that by D-Blondel and S-Blondel at $\alpha = 0.05$. Also, C-Blondel is comparable with D-Blondel and S-Blondel in terms of modularity during all snapshots. On average, the C-Blondel achieved 0.02 points improvement than D-Blondel in terms of modularity. Although the execution time of C-Blondel is slightly much more than that of D-Blondel, it achieved 14.4-s improvement than S-Blondel. It can be seen in Fig. 6 that C-Blondel underperforms D-Blondel and S-Blondel at $\alpha = 0.5$ and $\alpha = 0.6$ in terms of modularity. C-Blondel achieves 3.43- and 0.45-s

TABLE I
STATISTICAL INFORMATION OF THE ENRON EMAIL,
CIT-HEPTh, AND FACEBOOK DATA SETS

Dataset	Cit-HepTh	Enron	Facebook
Duration	Jan 1993 – Apr 2003	Jan 1993 – Apr 2003	Jan 1993 – Apr 2003
cc	0.292	0.402	0.218
$\langle \bar{d} \rangle$	18.663	7.704	14.617
#Edge	201099	325526	614797
#Node	21550	83910	61096

improvements than the S-Blondel and D-Blondel in terms of the execution time at $\alpha = 0.5$ (in average). Furthermore, C-Blondel achieves 3.45- and 0.50-s improvements than the S-Blondel and D-Blondel in terms of the execution time at $\alpha = 0.6$ (in average).

3) *Enron Email Data Set*: According to Fig. 7, the distribution degree of Enron network is power law. Given the use of the method proposed by Clauset *et al.* [42], the coefficient α in this data set is estimated to be 3.05. Therefore, by increasing the destruction parameter, the number of nodes inside the supernodes is expected to rise, and the size of the compressed graph decreases sharply.

As shown in Fig. 8, increasing in the amount of destruction parameter keeps fixed the execution time and modularity, and it does not change dramatically. The reason can be explained as follows. According to Table I, the average degree of nodes in this network is eight. Therefore, it can be concluded that the average degree of nodes in most communities is probably eight. If even the value of the destruction parameter is equal to one, 88.54% of nodes has a degree below the average. As a result, increasing the destruction parameter does not change the number of destructive nodes significantly.

As it can be observed from Fig. 9, C-Blondel outperforms S-Blondel and D-Blondel in terms of the execution time, but C-Blondel underperforms D-Blondel and S-Blondel in terms of modularity. C-Blondel achieves similar results with two other methods in terms of the community number. When the value of the power-law distribution parameter is high, the variation of nodes degree is high. Therefore, the algorithm is more sensitive to the destructive parameter. On the other hand, the experiments show that when the value of the power-law distribution parameter is low, the sensitivity of the proposed algorithm to the destructive parameter decreases, since the diversity of nodes degree decreases.

V. CONCLUSION

In this article, a new dynamic community detection algorithm is proposed based on the Louvain algorithm, which uses historical information of network. C-Blondel generates a compressed graph from the previous snapshot communities and network changes. The compressed graph is much denser and informative compared with the original graph. C-Blondel is also sensitive to change the value of the destruction parameter. This parameter indicates the level of the destructiveness of a node. The experiments indicated that C-Blondel outperforms

S-Blondel and D-Blondel in terms of the execution time. However, it is comparable with regard to the modularity criteria.

ACKNOWLEDGMENT

The authors would like to thank Dr. Alan Mislove at Northeastern University for providing the Facebook data set.

REFERENCES

- [1] J. He, D. Chen, C. Sun, Y. Fu, and W. Li, "Efficient stepwise detection of communities in temporal networks," *Phys. A, Stat. Mech. Appl.*, vol. 469, pp. 438–446, Mar. 2017.
- [2] M. Seifikar and S. Farzi, "A comprehensive study of online event tracking algorithms in social networks," *J. Inf. Sci.*, vol. 45, no. 2, pp. 156–168, Apr. 2019.
- [3] M. Girvan and M. E. J. Newman, "Community structure in social and biological networks," *Proc. Nat. Acad. Sci. USA*, vol. 99, no. 12, pp. 7821–7826, Jun. 2002.
- [4] M. A. Porter, J.-P. Onnela, and P. J. Mucha, "Communities in networks," *Notices AMS*, vol. 56, no. 9, pp. 1082–1097, 2009.
- [5] J. Duch and A. Arenas, "Community detection in complex networks using extremal optimization," *Phys. Rev. E, Stat. Phys. Plasmas Fluids Relat. Interdiscip. Top.*, vol. 72, p. 027104, Aug. 2005.
- [6] C. Pizzuti, "Evolutionary computation for community detection in networks: A review," *IEEE Trans. Evol. Comput.*, vol. 22, no. 3, pp. 464–483, Jun. 2018.
- [7] S. Fortunato, "Community detection in graphs," *Phys. Rep.*, vol. 486, nos. 3–5, pp. 75–174, Feb. 2010.
- [8] J. Xie and B. K. Szymanski, "LabelRank: A stabilized label propagation algorithm for community detection in networks," in *Proc. IEEE 2nd Netw. Sci. Workshop (NSW)*, Apr. 2013, pp. 138–143.
- [9] J. Chen, L. Chen, Y. Chen, M. Zhao, S. Yu, Q. Xuan, and X. Yang, "GA-based Q-attack on community detection," *IEEE Trans. Comput. Social Syst.*, vol. 6, no. 3, pp. 491–503, Jun. 2019.
- [10] N. Dakiche, F. B.-S. Tayeb, Y. Slimani, and K. Benatchba, "Tracking community evolution in social networks: A survey," *Inf. Process. Manage.*, vol. 56, no. 3, pp. 1084–1102, May 2019.
- [11] P. Agarwal, R. Verma, A. Agarwal, and T. Chakraborty, "DyPerm: Maximizing permanence for dynamic community detection," *Advances in Knowledge Discovery and Data Mining (Lecture Notes in Computer Science)*. Cham, Switzerland: Springer, 2018, pp. 437–449.
- [12] X. Niu, W. Si, and C. Q. Wu, "A label-based evolutionary computing approach to dynamic community detection," *Comput. Commun.*, vol. 108, pp. 110–122, Aug. 2017.
- [13] M. Cordeiro, R. P. Sarmiento, and J. Gama, "Dynamic community detection in evolving networks using locality modularity optimization," *Social Netw. Anal. Mining*, vol. 6, no. 1, p. 15, 2016.
- [14] N. P. Nguyen, T. N. Dinh, Y. Shen, and M. T. Thai, "Dynamic social community detection and its applications," *PLoS ONE*, vol. 9, no. 4, Apr. 2014, Art. no. e91431.
- [15] V. D. Blondel, J.-L. Guillaume, R. Lambiotte, and E. Lefebvre, "Fast unfolding of communities in large networks," *J. Stat. Mech., Theory Exp.*, vol. 2008, no. 10, Oct. 2008, Art. no. P10008.
- [16] D. Greene, D. Doyle, and P. Cunningham, "Tracking the evolution of communities in dynamic social networks," in *Proc. Int. Conf. Adv. Social Netw. Anal. Mining (ASONAM)*, Aug. 2010, pp. 176–183.
- [17] J. Kauffman, A. Kittas, L. Bennett, and S. Tsoka, "DyCoNet: A gephi plugin for community detection in dynamic complex networks," *PLoS ONE*, vol. 9, no. 7, Jul. 2014, Art. no. e101357.
- [18] H. Alvari, A. Hajibagheri, and G. Sukthankar, "Community detection in dynamic social networks: A game-theoretic approach," in *Proc. IEEE/ACM Int. Conf. Adv. Social Netw. Anal. Mining (ASONAM)*, Aug. 2014, pp. 101–107.
- [19] F. Folino and C. Pizzuti, "An evolutionary multiobjective approach for community discovery in dynamic networks," *IEEE Trans. Knowl. Data Eng.*, vol. 26, no. 8, pp. 1838–1852, Aug. 2014.
- [20] J. Xie, M. Chen, and B. K. Szymanski, "LabelRankT: Incremental community detection in dynamic networks via label propagation," in *Proc. Workshop Dyn. Netw. Manage. Mining (DyNetMM)*. New York, NY, USA: ACM, 2013, pp. 25–32.
- [21] Y.-R. Lin, Y. Chi, S. Zhu, H. Sundaram, and B. L. Tseng, "Facetnet: A framework for analyzing communities and their evolutions in dynamic networks," in *Proc. 17th Int. Conf. World Wide Web (WWW)*. New York, NY, USA: ACM, 2008, pp. 685–694.

- [22] N. P. Nguyen, T. N. Dinh, S. Tokala, and M. T. Thai, "Overlapping communities in dynamic networks: Their detection and mobile applications," in *Proc. 17th Annu. Int. Conf. Mobile Comput. Netw. (MobiCom)*. New York, NY, USA: ACM, 2011, pp. 85–96.
- [23] W. Xie, F. Zhu, J. Jiang, E.-P. Lim, and K. Wang, "TopicSketch: Real-time bursty topic detection from Twitter," *IEEE Trans. Knowl. Data Eng.*, vol. 28, no. 8, pp. 2216–2229, Aug. 2016.
- [24] Y. Xin, Z.-Q. Xie, and J. Yang, "An adaptive random walk sampling method on dynamic community detection," *Expert Syst. Appl.*, vol. 58, pp. 10–19, Oct. 2016.
- [25] Y. Xin, Z.-Q. Xie, and J. Yang, "The adaptive dynamic community detection algorithm based on the non-homogeneous random walking," *Phys. A, Stat. Mech. Appl.*, vol. 450, pp. 241–252, May 2016.
- [26] B. Klimt and Y. Yang, "The enron corpus: A new dataset for email classification research," in *Proc. Eur. Conf. Mach. Learn.*, Pisa, Italy. Berlin, Germany: Springer, 2004, pp. 217–226.
- [27] J. Leskovec, J. Kleinberg, and C. Faloutsos, "Graph evolution: Densification and shrinking diameters," *ACM Trans. Knowl. Discovery Data*, vol. 1, no. 1, p. 2, 2007.
- [28] B. Viswanath, A. Mislove, M. Cha, and K. P. Gummadi, "On the evolution of user interaction in Facebook," in *Proc. 2nd ACM Workshop Online Social Netw. (WOSN)*. New York, NY, USA: ACM, 2009, pp. 37–42.
- [29] P. De Meo, E. Ferrara, G. Fiumara, and A. Provetti, "Mixing local and global information for community detection in large networks," *J. Comput. Syst. Sci.*, vol. 80, no. 1, pp. 72–87, Feb. 2014.
- [30] S. Fortunato, V. Latora, and M. Marchiori, "Method to find community structures based on information centrality," *Phys. Rev. E, Stat. Phys. Plasmas Fluids Relat. Interdiscip. Top.*, vol. 70, no. 5, 2004, Art. no. 056104.
- [31] K. Yang, Q. Guo, and J.-G. Liu, "Community detection via measuring the strength between nodes for dynamic networks," *Phys. A, Stat. Mech. Appl.*, vol. 509, pp. 256–264, Nov. 2018.
- [32] F. Liu, D. Choi, L. Xie, and K. Roeder, "Global spectral clustering in dynamic networks," *Proc. Nat. Acad. Sci. USA*, vol. 115, no. 5, pp. 927–932, Jan. 2018.
- [33] F. Jiang and J. Xu, "Dynamic community detection based on game theory in social networks," in *Proc. IEEE Int. Conf. Big Data (Big Data)*, Oct. 2015, pp. 2368–2373.
- [34] X. Zhou, X. Zhao, and Y. Liu, "A multiobjective discrete bat algorithm for community detection in dynamic networks," *Appl. Intell.*, vol. 48, no. 9, pp. 3081–3093, Sep. 2018.
- [35] Z. Zhao, C. Li, X. Zhang, F. Chiclana, and E. H. Viedma, "An incremental method to detect communities in dynamic evolving social networks," *Knowl.-Based Syst.*, vol. 163, pp. 404–415, Jan. 2019.
- [36] L. Wu, Q. Zhang, K. Guo, E. Chen, and C. Xu, "Dynamic community detection method based on an improved evolutionary matrix," *Concurrency Comput., Pract. Exper.*, vol. 20, no. 4, pp. 14–53, 2019.
- [37] I. Messaoudi and N. Kamel, "A multi-objective bat algorithm for community detection on dynamic social networks," *Appl. Intell.*, vol. 49, no. 6, pp. 2119–2136, Jun. 2019.
- [38] M. E. J. Newman and M. Girvan, "Finding and evaluating community structure in networks," *Phys. Rev. E, Stat. Phys. Plasmas Fluids Relat. Interdiscip. Top.*, vol. 69, Feb. 2004, Art. no. 026113.
- [39] L. A. Adamic, R. M. Lukose, A. R. Puniyani, and B. A. Huberman, "Search in power-law networks," *Phys. Rev. E, Stat. Phys. Plasmas Fluids Relat. Interdiscip. Top.*, vol. 64, no. 4, 2001, Art. no. 046135.
- [40] S. Chen, Z.-Z. Wang, L. Tang, Y.-N. Tang, Y.-Y. Gao, H.-J. Li, J. Xiang, and Y. Zhang, "Global vs local modularity for network community detection," *PLoS ONE*, vol. 13, no. 10, Oct. 2018, Art. no. e0205284.
- [41] J. Moody, D. McFarland, and S. Bender-deMoll, "Dynamic network visualization," *Amer. J. Sociol.*, vol. 110, no. 4, pp. 1206–1241, 2005.
- [42] A. Clauset, C. R. Shalizi, and M. E. J. Newman, "Power-law distributions in empirical data," *SIAM Rev.*, vol. 51, no. 4, pp. 661–703, Nov. 2009.



Mahsa Seifkar received the bachelor's degree in computer engineering and the master's degree in artificial intelligence and robotics from the K. N. Toosi University of Technology, Tehran, Iran, in 2016 and 2018, respectively.

She is currently a Data Scientist at one of the best banks in Iran. Her current research interests include social network analysis, natural language processing, and big data.



Saeed Farzi received the Ph.D. degree in computer engineering from Tehran University, Tehran, Iran, in 2016.

He joined the Artificial Intelligence Department, K. N. Toosi University of Technology, Tehran, in 2017. His research interests include machine learning, information retrieval, and social network analysis.



Masoud Barati received the Ph.D. degree in computer science from Sherbrooke University, Sherbrooke, QC, Canada, in 2018.

He is currently a Research Associate with the School of Computer Science and Informatics, Cardiff University, Cardiff, U.K. His research interests include social networks, blockchain-based techniques, cybersecurity, formal methods, and service composition.